

🔍 Guia Rápido: Expressões Regulares (Regex)

Referência teórica e prática para validação de campos de formulário em JavaScript.

1. O que é uma Expressão Regular (Regex)?

Uma **Expressão Regular (Regex)** é um padrão de texto usado para verificar se o valor digitado pelo usuário em um formulário segue um formato esperado (por exemplo: "conter exatamente 11 dígitos" ou "conter um número com DDD válido").

Em JavaScript, declaramos uma Regex entre barras:

```
const padraoCpf = /^\\d{11}$/;
```

2. Tabela de Símbolos Essenciais (Cheat-Sheet)

Símbolo	Significado	Exemplo
<code>\\d</code>	Qualquer dígito numérico (0 a 9)	<code>/\\d/</code> (encontra qualquer número)
<code>\\D</code>	Qualquer caractere que NÃO é número	<code>/\\D/</code> (letras, símbolos, espaços)
<code>^</code> (Início)	O texto precisa começar aqui	<code>/^\\d/</code> (começa com número)
<code>\$</code> (Fim)	O texto precisa terminar aqui	<code>/\\d\$/</code> (termina com número)
<code>{n}</code>	Exatamente n repetições	<code>/\\d{11}/</code> (exatamente 11 números)
<code>{min,max}</code>	Intervalo de repetições	<code>/\\d{10,11}/</code> (entre 10 e 11 números)
<code>\\</code> (Escape)	Anula o superpoder de um caractere especial	<code>/\\. /</code> (procura o ponto literal <code>.</code>)

3. ⚠️ O que é o "Escape" (a barra invertida `\\`)?

Em Regex, alguns símbolos possuem significados especiais no motor de busca (por exemplo, o ponto `.` sozinho significa "qualquer caractere existente").

Se você precisar buscar um caractere especial de forma literal (como um ponto final ou parêntese), você deve usar a barra invertida `\\` antes dele para **escapar** o seu efeito:

- `\\.` → Procura exatamente o caractere ponto final (`.`).
- `\\(` e `\\)` → Procura os caracteres parênteses literais (`(` e `)`).
- `\\-` → Procura o caractere traço literal (`-`).

4. Como Validar no JavaScript: O Método `regex.test(valor)`

O método `.test()` recebe o texto digitado e retorna um valor booleano: `true` (se o texto atende ao padrão) ou `false` (se não atende):

```
const regexCpf = /^\\d{11}$/;

// Testes de validação:
console.log(regexCpf.test("12345678901")); // Retorna true (11 dígitos válidos)
```

```
console.log(regexCpf.test("12345")); // Retorna false (apenas 5 dígitos)
console.log(regexCpf.test("1234567890A")); // Retorna false (contém letra)
```

5. 📄 Exemplos de Padrões para Validação

A. CPF (Apenas 11 dígitos numéricos)

```
// Inicia com dígito (^), exige exatamente 11 números ({11}) e finaliza ($)
const regexCpf = /^\d{11}$/;
```

B. Telefone com DDD (10 ou 11 dígitos numéricos)

No Brasil, telefones fixos têm 10 dígitos (DDD + 8 números) e celulares têm 11 dígitos (DDD + 9 números):

```
// Inicia com dígito (^), aceita de 10 a 11 números ({10,11}) e finaliza ($)
// Exemplos aceitos: 85988887777 (celular) ou 8533334444 (fixo)
const regexTelefone = /^\d{10,11}$/;
```

C. E-mail (Formato básico: texto@texto.texto)

```
const regexEmail = /^[^\\s@]+@[^\\s@]+\\.\\.[^\\s@]+$/;
```

6. 🛠 Ferramentas Online para Testes

- [Regex101 \(regex101.com\)](https://regex101.com): Testador interativo que explica cada parte da sua Regex em tempo real.
- [RegExr \(regexr.com\)](https://regexr.com): Editor visual com cheat-sheet e testes instantâneos.